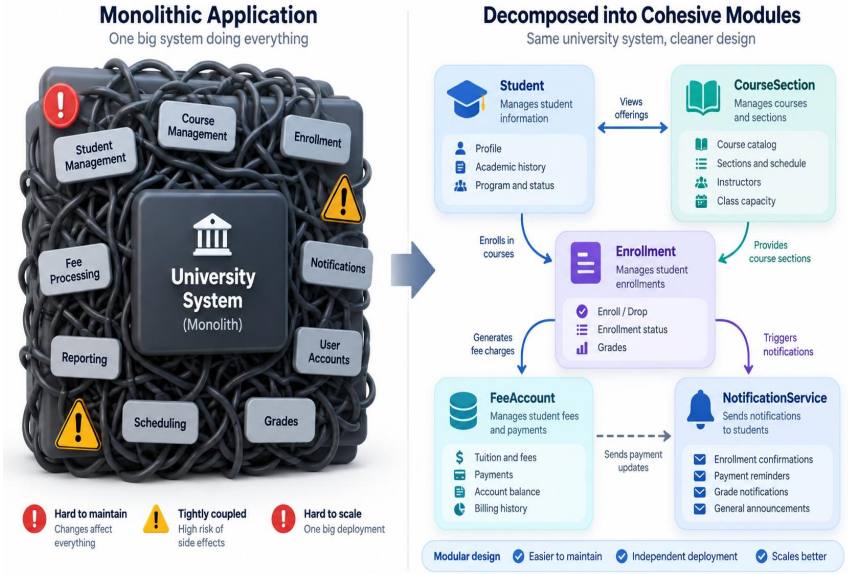
CSC241 — Object-Oriented Programming

Lecture 13 — Decomposition & Class Hierarchy Design

Breaking large systems into cohesive objects, assigning responsibilities, and designing meaningful abstractions.



Dr Muhammad Shahid Bhatti
Assistant Professor
Department of Computer Science
Office H-17
GitHub: github.com/mshahidbhatti/CSC241-OOP

Decomposition | Responsibilities | Information Expert | Low Coupling | Hierarchy Design | Refactoring

Bridge from Lecture 12

Last time we moved between UML and Java. Today we improve the design before inheritance syntax begins.

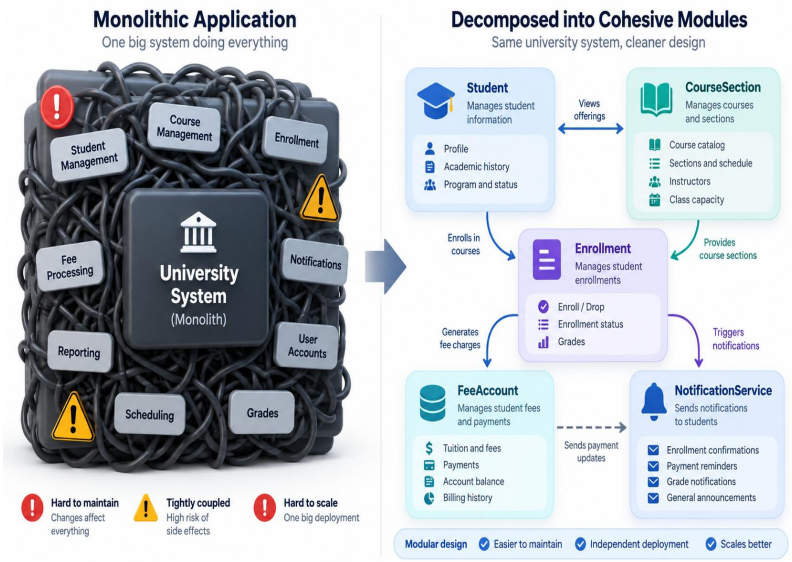
L13

Lecture 12 asked: “How do I map a design into code — and recover a design from code?”

Lecture 13 asks: “Is this decomposition sensible, cohesive, and maintainable?”

We will review responsibilities, collaborators, coupling, cohesion, hierarchy choices, and refactoring.

The goal is not to create more classes; the goal is to create better boundaries.



Think ahead

A weak decomposition produces fragile inheritance later. A strong design makes the upcoming inheritance lecture much easier to reason about.

Learning outcomes

By the end of the lecture, students should be able to:

decompose a large problem into cooperating classes with clear responsibilities;

assign behavior using information-expert reasoning;

explain cohesion and coupling as class-design criteria;

distinguish stable generalization opportunities from composition-based designs;

review a class hierarchy for semantic coherence;

refactor an overly large “God class” into focused collaborators.

Live-coding emphasis

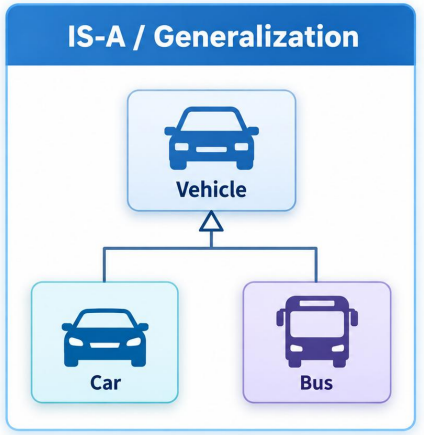
We will refactor a deliberately oversized UniversitySystem class into smaller classes and recompile after each step.

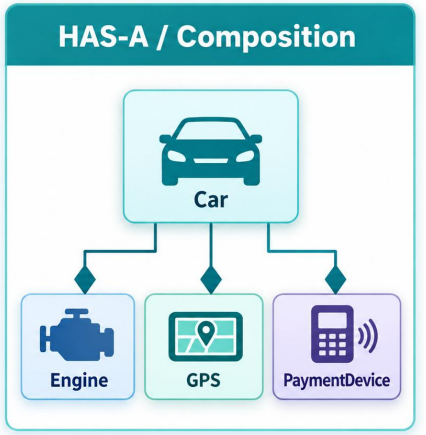
Repository path

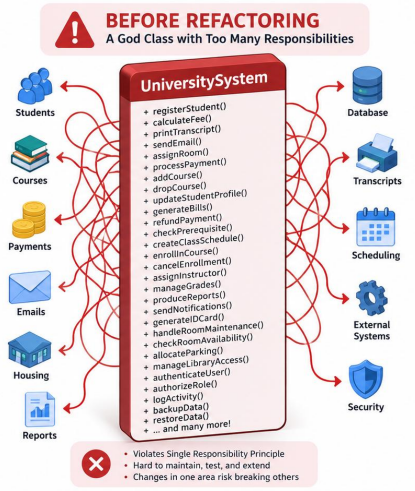
00_Course_Master\01_Lectures\Week07\L13\LiveCoding\01_Decomposition

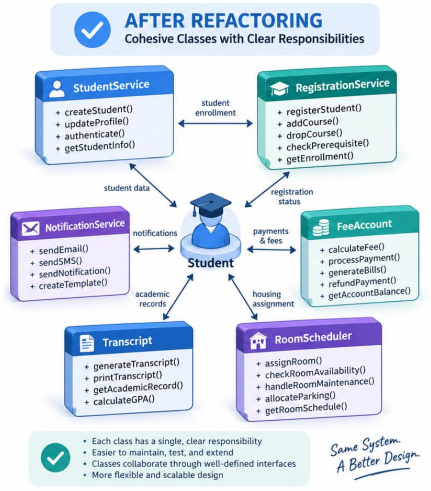
Lecture roadmap

1. Why decomposition matters
2. Responsibilities, cohesion, and coupling
3. Information Expert and coordination logic
4. Designing class hierarchies
5. Generalization vs composition decisions
6. Refactoring a God class
7. UML grouping, review checklist, and design activity









What is software decomposition?

Break a complex system into smaller units that can reason, change, and collaborate independently.

Each class should represent a focused concept or responsibility.

Boundaries should reduce accidental dependencies.

Objects cooperate through clear public operations rather than shared internal details.

A good decomposition makes changes local: one requirement should not force edits everywhere.

Decomposition is a design activity — not simply “split every method into another class.”

Design question

If a requirement changes, how many classes must you understand and edit?

Smell

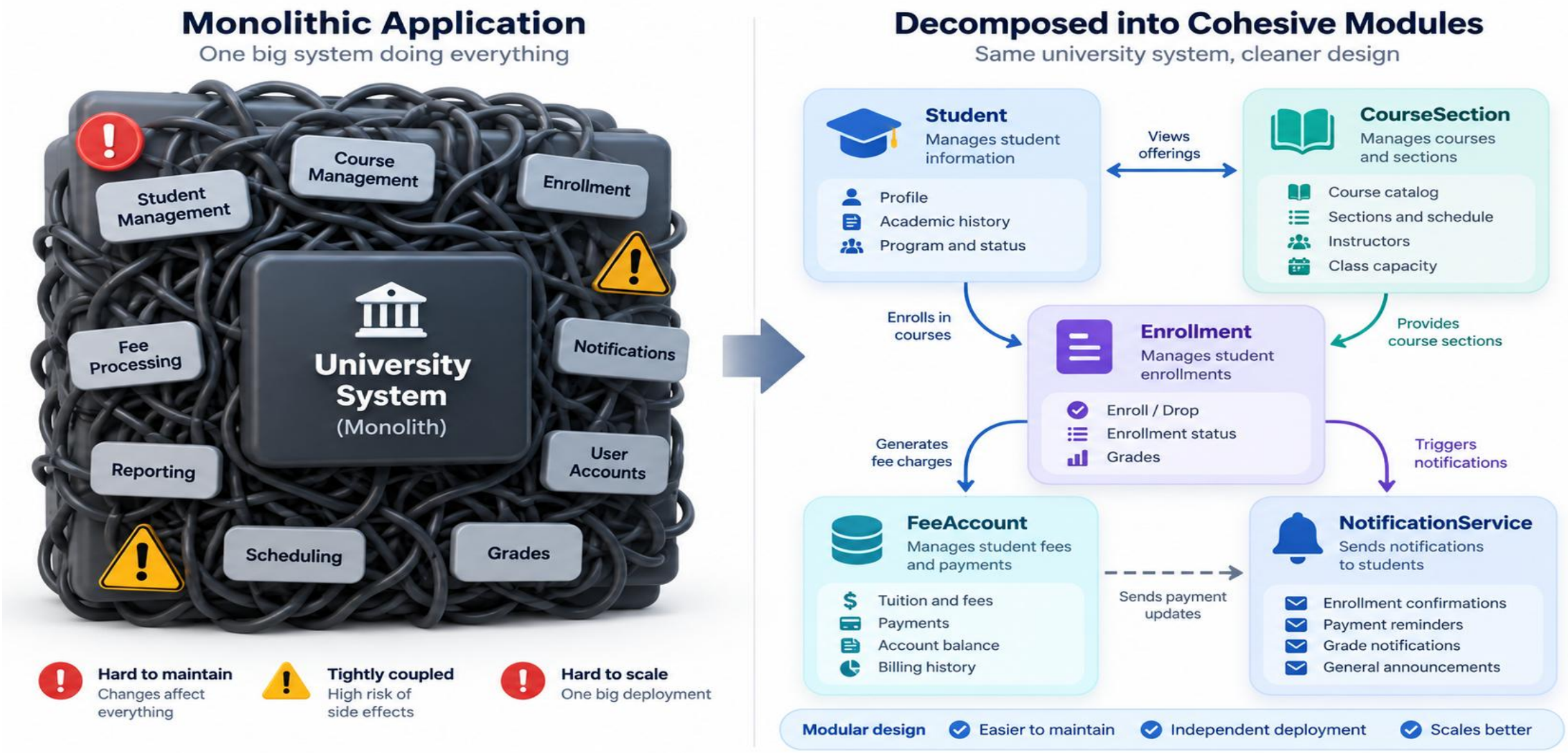
If one class knows students, fees, schedules, reports, email templates, rooms, and database details, the design likely needs decomposition.

© Dr Muhammad Shahid Bhatti

Visual — from tangled monolith to cohesive collaborators

L13

The same system becomes easier to reason about when responsibilities have clear homes.



A design smell: the “Overloaded class”

Before decomposition

```
public class UniversitySystem {
    void registerStudent(...) { ... }
    void enrollCourse(...) { ... }
    double calculateFee(...) { ... }
    void processPayment(...) { ... }
    void sendEmail(...) { ... }
    void assignRoom(...) { ... }
    void printTranscript(...) { ... }
    void createSchedule(...) { ... }
    // ...and much more
}
```

Too many unrelated reasons to change.

Large public API with weak conceptual focus.

Difficult to test one responsibility in isolation.

Small changes risk breaking unrelated behavior.

Teams cannot work independently without conflicts.

Important

A large class is not automatically bad. The problem is unrelated responsibilities and excessive knowledge of the whole system.

Where should we cut the system?

Useful decomposition boundaries often appear around domain concepts and responsibility clusters.

Student profile, program, academic status

CourseSection capacity, schedule, offered course

Enrollment links student to section; grade/status

FeeAccount charges, balance, payments

Notification messages and delivery

Transcript academic record presentation

Heuristic

Group things that change together. Separate things that change for different reasons.

Responsibility assignment: what should each object know and do?

What an object KNOWS

- Its own state and invariants
- References to collaborators it legitimately owns or uses
- Rules naturally associated with its concept
- Information required to answer its public questions

What an object DOES

- Protects and updates its state safely
- Performs calculations using information it owns
- Delegates work to appropriate collaborators
- Exposes a small, meaningful public API

Information Expert — assign responsibility where the needed information lives

Ask: “Which object already has the information needed to perform this responsibility?”

Place behavior close to the data and rules it depends on.

Avoid central controller classes that repeatedly pull data out of other objects and compute everything themselves.

The result is usually better cohesion and fewer cross-object dependencies.

Example question

Who knows whether CourseSection still has capacity?

Information Expert answer

```
class CourseSection {
    private int capacity;
    private int registered;

    boolean hasSeat() {
        return registered < capacity;
    }
}
```

Worked example — who should calculate remaining seats?

Client calculates it

```
// weak coordination-heavy approach
int remaining = section.getCapacity()
              - section.getRegistered();
```

CourseSection answers

```
// stronger object responsibility
int remaining = section.remainingSeats();
```

CourseSection already owns capacity and registered count.

It can preserve invariants and change its internal representation later.

Clients ask a meaningful question instead of reconstructing the rule.

This is a small example of information hiding + information expert.

Low coupling — minimize unnecessary knowledge between classes

Coupling measures how strongly one class depends on other classes or their internals.

Some coupling is unavoidable: objects must collaborate.

The goal is controlled coupling through narrow, stable interfaces.

Changes in one class should not force widespread changes in unrelated classes.

Avoid “train wreck” navigation chains that cross many objects.

High coupling smell

```
// brittle navigation chain
student.getAccount()
       .getBillingProfile()
       .getCurrentInvoice()
       .applyDiscount(0.10);
```

Better

Ask a closer collaborator to perform a meaningful operation instead of exposing a long internal object graph.

Refactoring for lower coupling

Expose one meaningful operation

```
class Student {
    private FeeAccount feeAccount;

    void applyScholarship(double rate) {
        feeAccount.applyDiscount(rate);
    }
}
```

The client no longer traverses several billing objects.

FeeAccount retains the fee/discount rule; Student only delegates.

Alternatively, a billing service can handle the use case.

Choose the entry point that best matches responsibilities.

Caution

Do not turn every class into a forwarding wrapper. Delegate only when it improves responsibility boundaries and reduces unnecessary exposure.

High cohesion — keep related responsibilities together

High cohesion

CourseSection manages capacity, registration state, schedule.

FeeAccount manages charges, balance, payment operations.

Methods support one coherent concept.

Easy to name the class and explain its purpose.

Low cohesion

Student class also sends emails, creates PDFs, runs SQL, calculates fees.

Many unrelated reasons to change.

Hard to explain the class in one sentence.

Often becomes a “dumping ground.”

Cohesion and coupling together

Aim for high cohesion inside classes and controlled/low coupling between classes.

High cohesion **Low coupling**	Preferred zone
High cohesion **High coupling**	Focused classes, but too dependent
Low cohesion **Low coupling**	Isolated but poorly focused
Low cohesion **High coupling**	Worst zone: tangled + unfocused

Domain logic vs coordination logic

A useful decomposition separates “what the domain means” from “how a use case is orchestrated.”

Domain objects

Student, CourseSection, Enrollment, FeeAccount

Own state, invariants, and domain rules

Examples: hasSeat(), applyPayment(), assignGrade()

Should remain understandable without UI/database details

Coordination objects

RegistrationService / Controller-like object

Coordinates several domain objects for a use case

Should not steal rules that belong to domain experts

Calls collaborators in the correct order

Live coding checkpoint 1 — refactor UniversitySystem

Start with a simple UniversitySystem containing enrollment, fees, and notification methods.

Identify rules owned by CourseSection and FeeAccount.
Extract one class and its fields at a time.
Make RegistrationService coordinate the enrollment use case.
Recompile and compare behavior after every extraction.
Update the UML class diagram alongside the Java files.

Compile incrementally

```
javac Student.java CourseSection.java Enrollment.ja
javac FeeAccount.java NotificationService.java
javac RegistrationService.java UniversityDemo.java
java UniversityDemo
```

Ask students

After each extraction: “Which class now owns this rule? Did coupling increase or decrease?”

13

Part II — Class Hierarchy Design

Generalize only when several concepts share a stable semantic abstraction.

What is a class hierarchy?

A hierarchy organizes types from more general concepts to more specialized concepts.

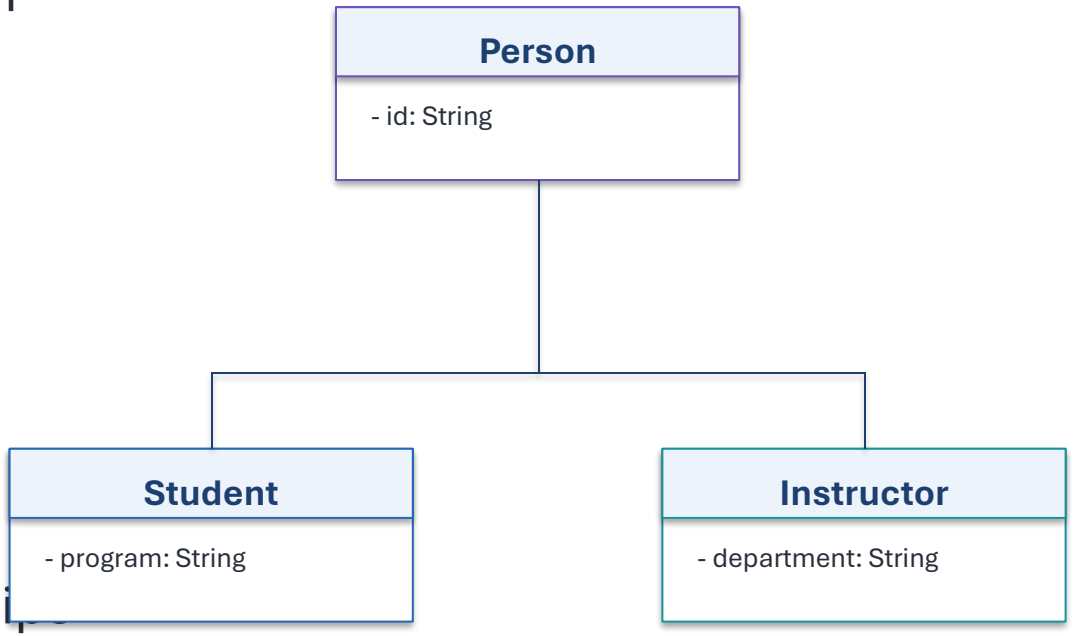
A superclass/general concept captures common meaning shared by subclasses/specializations.

A hierarchy should express an actual “is-a” relationship.

Shared code is a possible benefit, but semantic modeling comes first.

Avoid inventing a superclass only because two classes happen to share one field or method.

The next lecture will implement these relationships using Java inheritance syntax.



Look for stable abstractions — not superficial similarity

Stronger hierarchy candidate

Student and Instructor are both Persons.

Shared identity/contact concepts have common meaning.

Clients may reasonably talk about “a Person.”

Specializations add role-specific state/behavior.

Weak hierarchy candidate

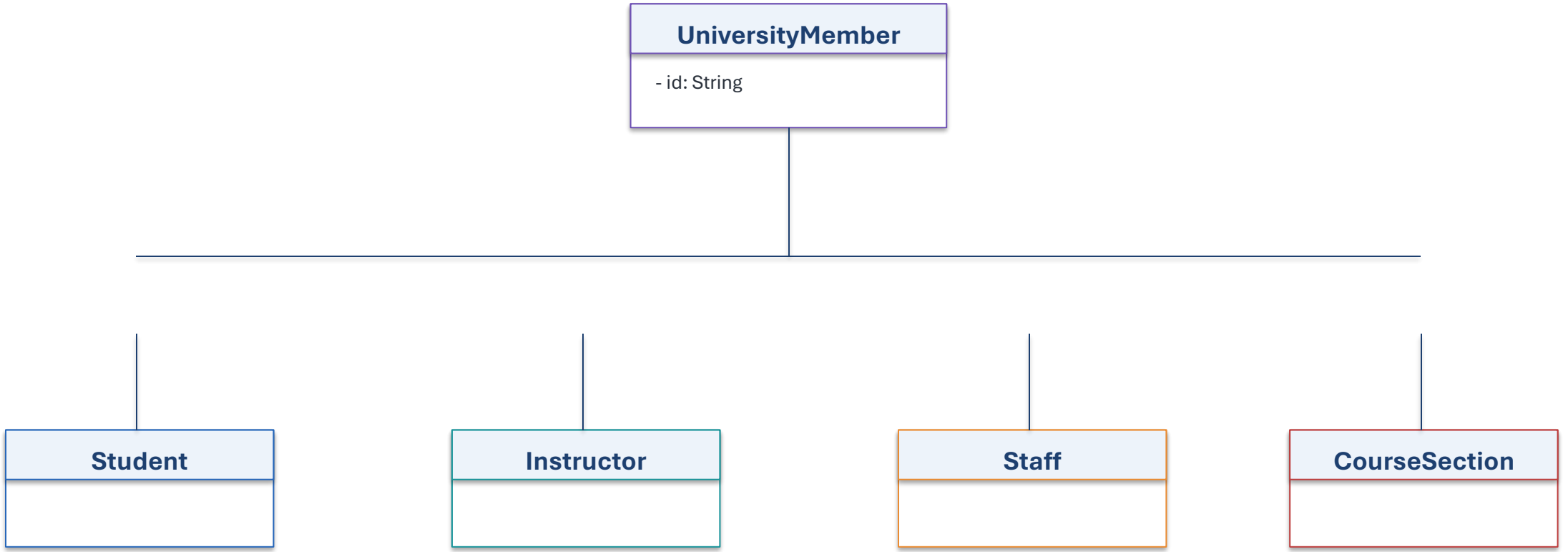
Car and Invoice both have an id.

Student and Printer both have a “status.”

Common fields alone do not establish an is-a relationship.

Forced parents create misleading abstractions.

Evaluate a proposed UML generalization tree



Challenge

Which child clearly does NOT belong in this hierarchy? Explain using “is-a” semantics, not shared fields.

Hierarchy design questions to ask before generalizing

Can I truthfully say “Every subclass is a superclass” in the domain?

Do subclasses share a stable concept — not only implementation details?

Would clients benefit from treating different subclasses through the common abstraction?

Are common responsibilities truly common, or likely to diverge?

Is the hierarchy shallow and understandable?

Would composition model the relationship more accurately?

Rule of thumb

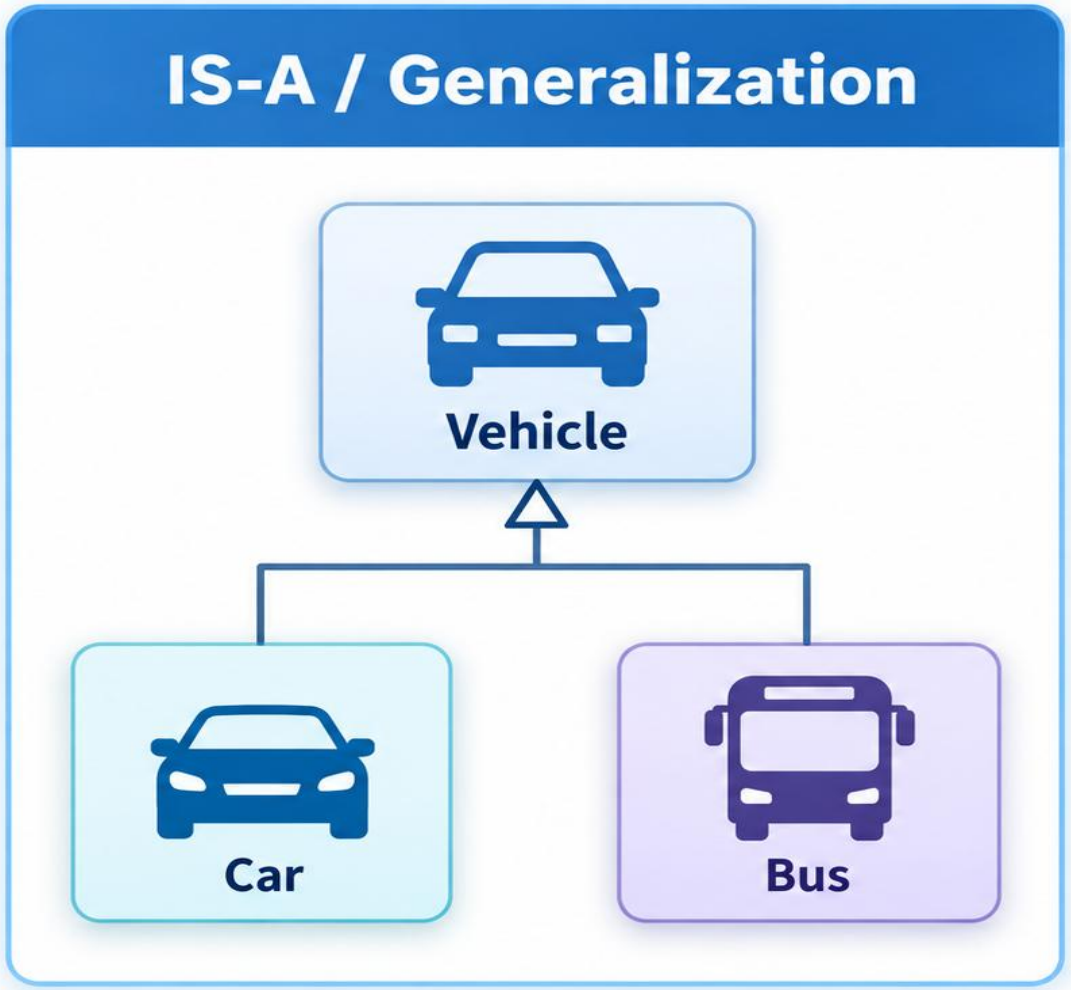
Model the domain truth first. Reuse is secondary.

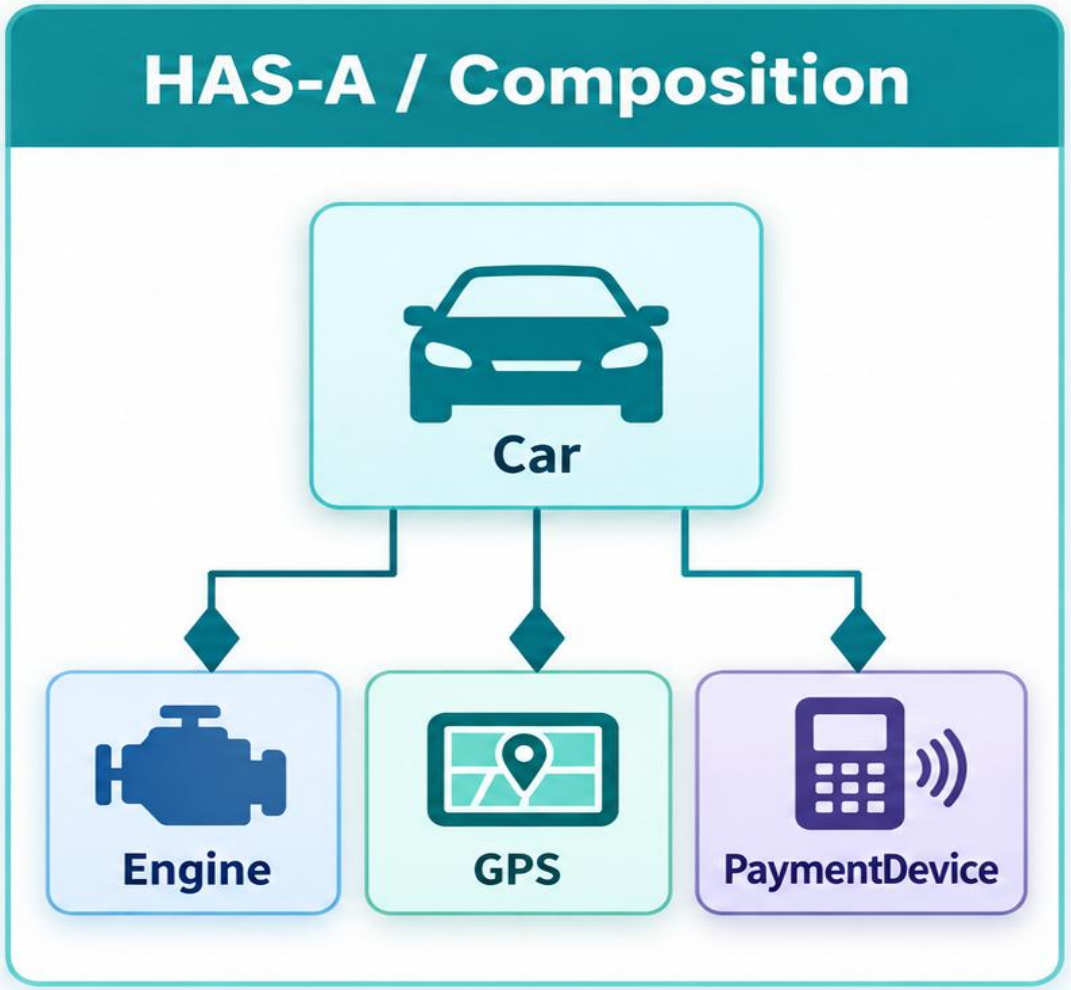
Preview

In Lecture 14, Java extends and super will implement the hierarchy. Today we decide whether the hierarchy deserves to exist.

Visual — generalization (IS-A) vs composition (HAS-A)

Many poor hierarchies disappear when you ask whether the relationship is really “is-a.”





Generalization or composition? — decision guide

Prefer generalization when...

A subtype is genuinely a specialized form of the parent concept.

The common abstraction is meaningful to clients.

Shared behavior expresses the same semantic contract.

Specializations can safely inherit the conceptual expectations.

Prefer composition when...

One object contains or uses another object.

You want to combine independent capabilities.

The relationship is “has-a” rather than “is-a.”

You want easier replacement of one collaborator with another.

Example — Car, Engine, and PaymentDevice

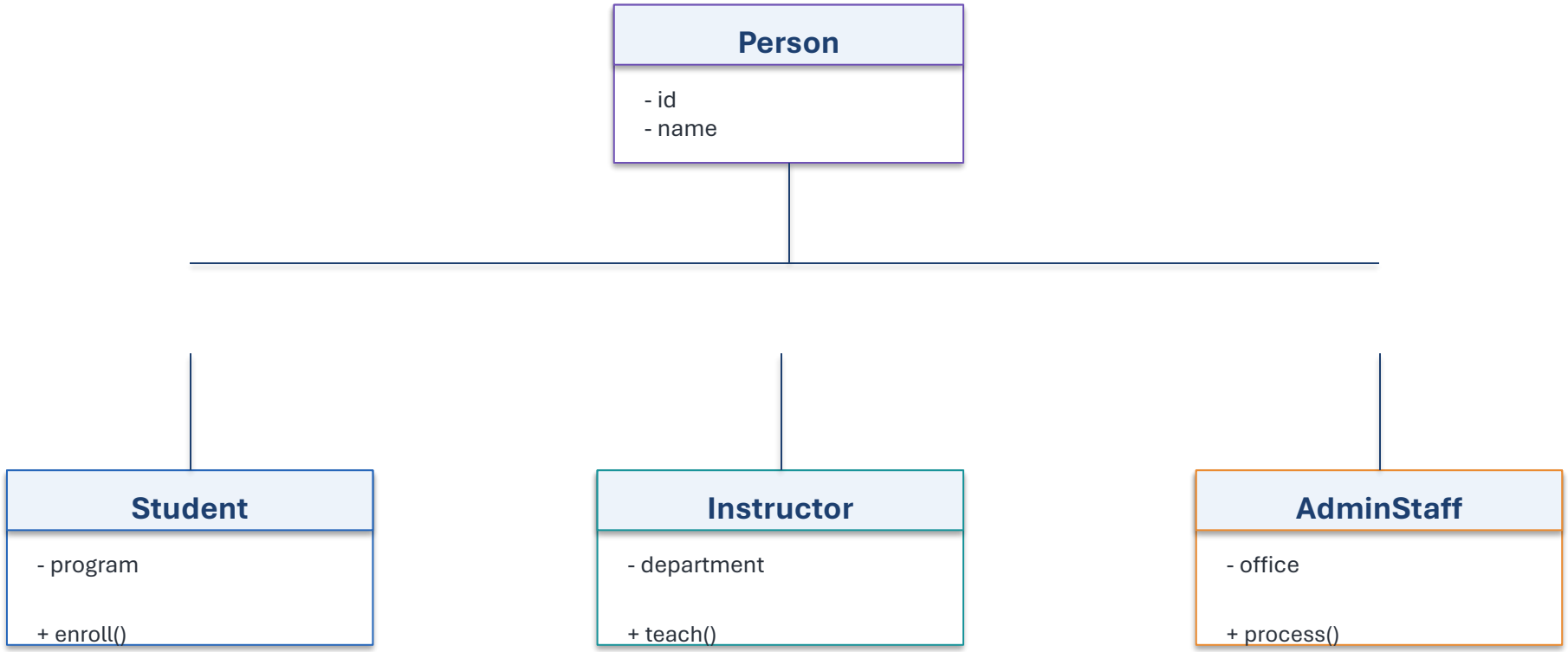
Composition-based design

```
class Car {
    private Engine engine;
    private GPS gps;
    private PaymentDevice paymentDevice;

    void start() { engine.start(); }
}
```

A Car may collaborate with replaceable components without becoming a subtype of those components.

Example — a meaningful domain hierarchy (conceptual only)



Question

What should stay in Person, and what should remain role-specific? Move only stable common responsibilities upward.

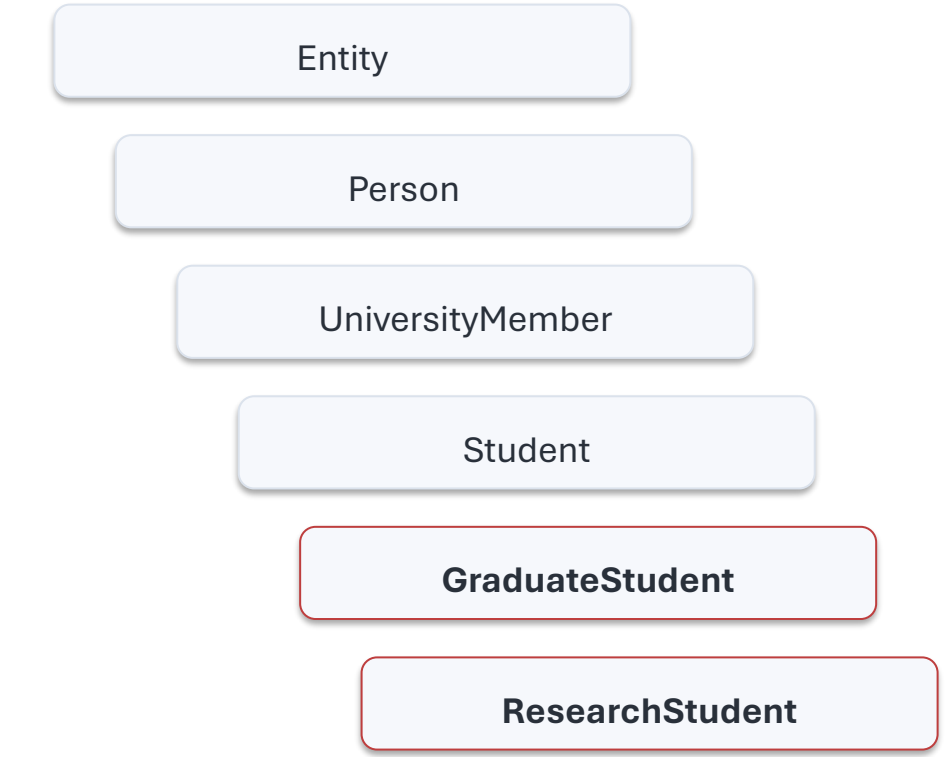
Avoid overly deep or semantically weak hierarchies

Every inheritance level adds assumptions that subclasses must understand.

Deep chains make constructor behavior, protected state, and overriding harder to reason about.

A hierarchy built mainly for code reuse can become fragile when business rules diverge.

Prefer shallow, meaningful abstractions and use composition for independent variation.

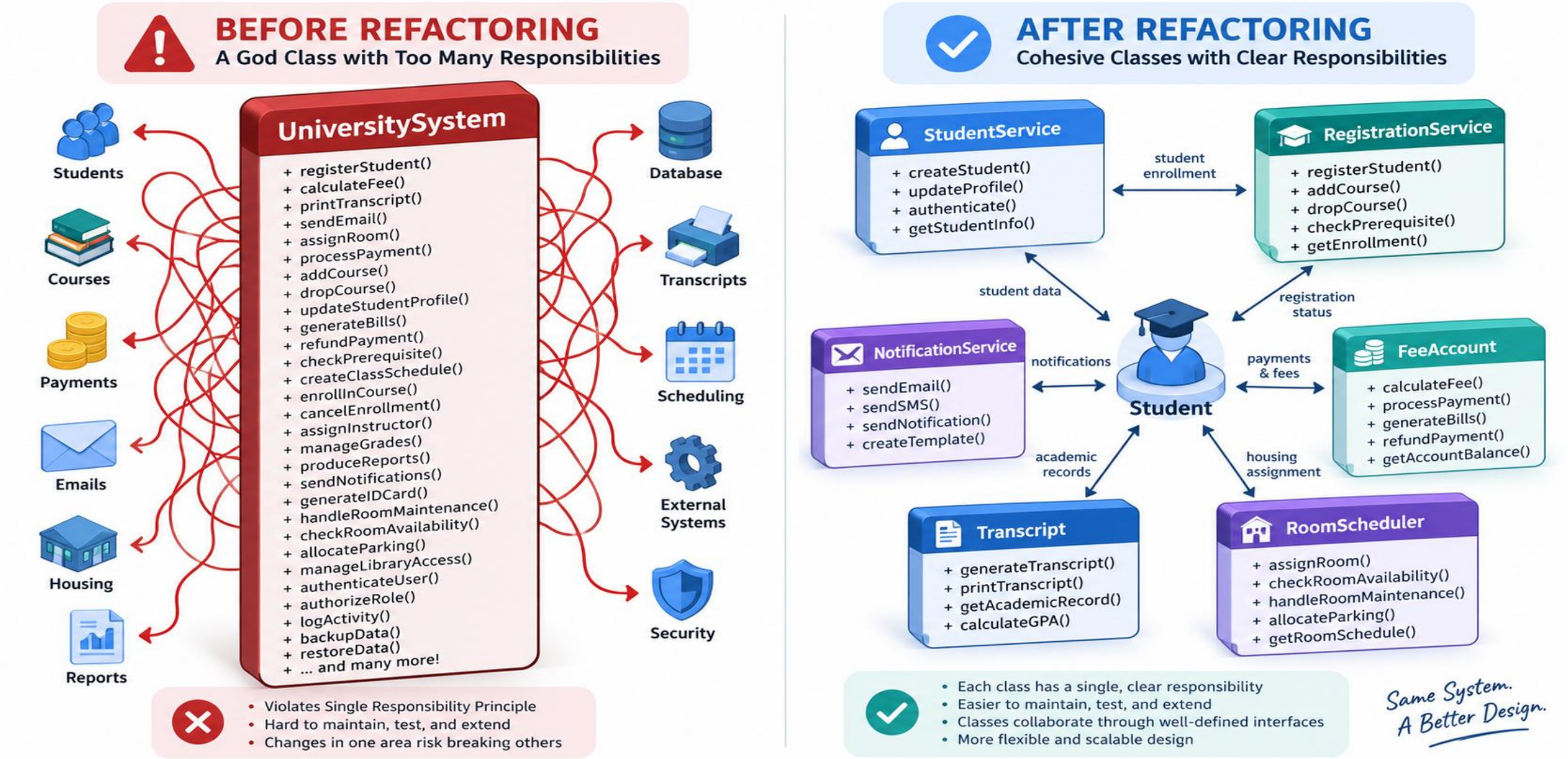


Smell

If you need several arrows just to explain what an object "really is," reconsider the abstraction.

Visual — refactoring a God class into cohesive collaborators

Refactoring changes structure while preserving behavior.



Refactoring strategy: split by reason to change

List all fields and methods of the oversized class.

Group related state and behavior into responsibility clusters.

Choose a clear name for each cluster/class.

Move methods together with the state they need.

Introduce collaborator references only where required.

Redirect calls, compile, and run tests after each move.

Review cohesion and coupling again after the split.

Calls evolve with the design

```
student.payFee(amount);
```

Safety

Refactor in small compilable steps. Structure first; behavior should remain equivalent.

Live coding checkpoint 2 — refactor and trace collaboration

Client after decomposition

```
Student s = new Student("BAI-005", "Abeer Amina");
CourseSection oop = new CourseSection("CSC241-A", 40);
RegistrationService service = new RegistrationService();

service.enroll(s, oop);
```

Trace which object owns the seat-capacity rule.

Trace which object records the enrollment relationship.

Ask whether RegistrationService is coordinating or stealing domain logic.

Intentionally move one rule to the wrong place and discuss the design smell.

Compile and run after each modification.

Organizing related classes into packages / conceptual groups

Grouping makes subsystem boundaries visible and helps expose unwanted cross-module dependencies.

registration

- Student
- CourseSection
- Enrollment
- RegistrationService

finance

- FeeAccount
- Payment
- Invoice

communication

- NotificationService
- EmailMessage

Dependency check

If finance classes repeatedly reach into registration internals, your package boundaries may be wrong or your APIs too leaky.

Cross-module dependency review

Healthy dependency

Registration asks FeeAccount for a payment-related operation.

Communication receives a small message/notification request.

Modules expose narrow public contracts.

Internal details remain private.

Warning signs

Every module imports every other module.

UI directly manipulates deep domain fields.

Finance must know CourseSection internals.

Changes ripple across packages.

Class design review checklist

Can I explain each class's responsibility in one sentence?

Does each class own the information needed for its main behavior?

Are unrelated reasons to change separated?

Are collaborators minimal and meaningful?

Is the "is-a" relationship semantically true for every proposed subclass?

Would composition be clearer than generalization?

Is the hierarchy shallow and coherent?

Can I test important responsibilities without constructing half the system?

Use this on your own projects

Before adding another class, method, or inheritance edge, run the checklist.

Design activity — Campus Event Registration

Work in groups for 12–15 minutes.

A campus event system lets students register for events. Each event has a capacity and venue; some events require a fee. Successful registration creates a record and triggers confirmation. Staff can request an attendance list.

- Identify 5–7 candidate classes.
- Assign one primary responsibility to each.
- Mark likely collaborators.
- Choose where capacity checking belongs using Information Expert.
- Identify one possible hierarchy and decide whether it is genuinely useful.
- Find one relationship that should clearly use composition/association instead of inheritance.

Deliverable

Sketch a compact UML class diagram and be prepared to defend two design decisions.

Activity debrief — one reasonable decomposition

There is more than one acceptable design; judge it by responsibility clarity, cohesion, and coupling.



Information Expert choice

Event should normally own the capacity rule because it knows capacity and current registrations.

Coordination choice

A service may coordinate registration + fee + notification, but it should delegate each domain rule to the expert object.

Quiz #2 readiness — quick concept check

1. What is the difference between cohesion and coupling?
2. Which class is the Information Expert for section capacity?
3. Why is a God class difficult to maintain?
4. Give one example where composition is better than generalization.
5. What makes a superclass abstraction semantically coherent?
6. How can a package diagram reveal design problems?

Classroom use

Ask students to answer first without notes. Then discuss reasoning rather than memorized definitions.

Lecture recap & exit ticket

Strong OO design is mostly about responsibility boundaries.

L13

Remember

Decompose by cohesive responsibilities.

Put behavior with the object that has the information.

Keep coupling controlled and explicit.

Generalize only for a real, stable IS-A relationship.

Use composition for HAS-A collaboration.

Refactor God classes incrementally.

Exit ticket

Name one class you would extract from a God class and why.

Explain Information Expert in your own words.

Give one high-coupling design smell.

What question helps decide generalization vs composition?

Why can deep hierarchies become fragile?

Next lecture

Inheritance — single, multilevel, and hierarchical inheritance; extends; super; and constructor behavior in Java.